Session 8: Functions — Notes & Code (C Language)

Mastering Modular Programming, Pointers, and Scope in C

October 11, 2026

Contents

1 Key Concepts **2**

2 Task 1: `getUserInitials` — Return Initials in Uppercase **2**
2.1 Notes . . . 2
2.2 Code . . . 2
2.3 Output . . . 3

3 Task 2: `addToCart` — Pass Cart Array by Reference **4**
3.1 Notes . . . 4
3.2 Code . . . 4
3.3 Output . . . 4

4 Task 3: Pass-by-Value vs Pass-by-Reference (Followers) **5**
4.1 Notes . . . 5
4.2 Code . . . 5
4.3 Output . . . 5

5 Task 4: `formatPrice` — Flipkart-Style Price Tag (Rs. 1,599) **6**
5.1 Notes . . . 6
5.2 Code . . . 6
5.3 Output . . . 6

6 Task 5: Refactor — Reusable `capitalizeFirst` for ANY String **8**
6.1 Notes . . . 8
6.2 Code . . . 8
6.3 Output . . . 8

7 Quick Revision Summary **10**

1 Key Concepts

- **Declaration (prototype):** tells the compiler the function's name, return type, and parameters—e.g., `int add(int a, int b);`
- **Definition:** the actual body of the function.
- **Calling:** using the function in `main()` or elsewhere.
- **Pass-by-value:** a *copy* of the variable is sent—changes inside the function do NOT affect the original.
- **Pass-by-reference (C uses pointers):** the *address* is sent—changes DO affect the original. Arrays in C always decay to pointers, so they behave like pass-by-reference automatically.

2 Task 1: `getUserInitials` — Return Initials in Uppercase

2.1 Notes

- Function takes a full name string, extracts the **first letter of each word**, converts to uppercase, and returns the initials.
- Use `isupper()`/`toupper()` from `<ctype.h>`.
- Since a function can't return a local array safely, either pass an output buffer or return a `static` string. Best practice: pass a result buffer.

2.2 Code

```c
#include <stdio.h>
#include <ctype.h>

// Declaration
void getUserInitials(char name[], char result[]);

// Definition
void getUserInitials(char name[], char result[]) {
    int k = 0;
    result[k++] = toupper(name[0]);          // first letter of first word
    for (int i = 1; name[i] != '\0'; i++) {
        if (name[i - 1] == ' ') {            // letter after a space = new
            word
            result[k++] = toupper(name[i]);
        }
    }
    result[k] = '\0';
}

int main() {
    char initials[10];
    getUserInitials("Virat Kohli", initials);
    printf("Initials: %s\n", initials);      // VK
```

```
    getUserInitials("ms dhoni", initials);
    printf("Initials: %s\n", initials);       // MD
    return 0;
}
```

Listing 1: Extracting Initials in C

2.3 Output

```
Initials: VK
Initials: MD
```

3 Task 2: `addToCart` — Pass Cart Array by Reference

3.1 Notes

- In C, when you pass an array to a function, you actually pass a **pointer** to its first element—so changes **persist outside** the function.
- We must also pass `*count` (pointer to the count) so the updated size is visible in `main()` too.
- After adding, loop through the cart to print it.

3.2 Code

```c
#include <stdio.h>
#include <string.h>

// cart is passed as a pointer (reference behavior), count as int*
void addToCart(char cart[][50], int *count, char product[]) {
    strcpy(cart[*count], product);      // add at end
    (*count)++;                         // increase cart size
    printf("Added '%s' to cart.\n", product);
}

void printCart(char cart[][50], int count) {
    printf("[Cart] Cart contents:\n");
    for (int i = 0; i < count; i++) {
        printf("  %d. %s\n", i + 1, cart[i]);
    }
}

int main() {
    char cart[10][50];
    int count = 0;

    addToCart(cart, &count, "iPhone 15");
    addToCart(cart, &count, "Nike Shoes");
    addToCart(cart, &count, "Bluetooth Speaker");

    printCart(cart, count);
    printf("Total items: %d\n", count);    // changes persisted! count = 3
    return 0;
}
```

Listing 2: Cart Management with Pointers

3.3 Output

```
Added 'iPhone 15' to cart.
Added 'Nike Shoes' to cart.
Added 'Bluetooth Speaker' to cart.
[Cart] Cart contents:
  1. iPhone 15
  2. Nike Shoes
  3. Bluetooth Speaker
Total items: 3
```

4 Task 3: Pass-by-Value vs Pass-by-Reference (Followers)

4.1 Notes

- `increaseFollowersByValue(int n)`: receives a **copy**—the original followers count stays the same.
- `increaseFollowersByReference(int *n)`: receives the **address**— `*n += 1000` modifies the original variable.
- This is the classic demonstration of the two passing styles.

4.2 Code

```c
#include <stdio.h>

// Pass-by-VALUE: works on a copy
void increaseFollowersByValue(int n) {
    n += 1000;
    printf("Inside byValue function: %d\n", n);
}

// Pass-by-REFERENCE: works on the original via pointer
void increaseFollowersByReference(int *n) {
    *n += 1000;
    printf("Inside byReference function: %d\n", *n);
}

int main() {
    int followers = 5000;

    increaseFollowersByValue(followers);
    printf("After byValue call:  %d  (unchanged [X])\n\n", followers);

    increaseFollowersByReference(&followers);   // pass the address
    printf("After byReference call: %d  (changed [OK])\n", followers);

    return 0;
}
```

Listing 3: Pass-by-Value vs Pass-by-Reference Demonstration

4.3 Output

```
Inside byValue function: 6000
After byValue call:  5000  (unchanged [X])

Inside byReference function: 6000
After byReference call: 6000  (changed [OK])
```

5 Task 4: `formatPrice` — Flipkart-Style Price Tag (Rs. 1,599)

5.1 Notes

- Formats an integer price with the **Indian comma system**: last 3 digits grouped, then every 2 digits (1,59,999).
- Approach: convert digits from the right, insert commas at the right positions, build the string **in reverse**, then reverse it at the end.
- The function "returns a string" by writing into a caller-provided buffer (standard C practice).

5.2 Code

```c
#include <stdio.h>
#include <string.h>

void formatPrice(int price, char result[]) {
    char digits[20], temp[25];
    sprintf(digits, "%d", price);          // number -> string

    int len = strlen(digits);
    int k = 0, pos = 0;

    // Build from right to left with commas (Indian style)
    for (int i = len - 1; i >= 0; i--) {
        temp[k++] = digits[i];
        pos++;
        // comma after last 3 digits, then after every 2
        if (i > 0 && (pos == 3 || (pos > 3 && (pos - 3) % 2 == 0)))
            temp[k++] = ',';
    }
    temp[k] = '\0';

    // Reverse temp into result
    int n = strlen(temp);
    for (int i = 0; i < n; i++)
        result[i] = temp[n - 1 - i];
    result[n] = '\0';
}

int main() {
    char price[25];

    formatPrice(1599,    price); printf("Phone    : Rs.%s\n", price);
    formatPrice(29999,   price); printf("Laptop   : Rs.%s\n", price);
    formatPrice(1234567, price); printf("Car      : Rs.%s\n", price);
    return 0;
}
```

Listing 4: Indian Number System Price Formatting

5.3 Output

```
Phone    : Rs.1,599
```

```
Laptop  : Rs.29,999
Car     : Rs.12,34,567
```

6 Task 5: Refactor — Reusable `capitalizeFirst` for ANY String

6.1 Notes

- **Refactoring** = rewriting a function so it's generic and reusable instead of tied to one use case.
- Instead of a function that only handles product names or usernames, write `capitalizeFirst(char str[])` that capitalizes the first letter of **any** string (and after spaces too, for multi-word names).
- Works for product names, usernames, full names, cities—anything.

6.2 Code

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

// REUSABLE: capitalizes the first letter of any string
// (and the first letter of every word after a space)
void capitalizeFirst(char str[]) {
    int newWord = 1;   // start of string counts as a new word
    for (int i = 0; str[i] != '\0'; i++) {
        if (newWord && isalpha(str[i])) {
            str[i] = toupper(str[i]);
            newWord = 0;
        }
        if (str[i] == ' ')
            newWord = 1;
    }
}

void printFormatted(char label[], char str[]) {
    capitalizeFirst(str);   // modifies in place (array = pass-by-reference
        )
    printf("%s: %s\n", label, str);
}

int main() {
    char product[] = "iphone 15 pro";
    char username[] = "rahul sharma";
    char city[] = "mumbai";

    printFormatted("Product ", product);
    printFormatted("Username", username);
    printFormatted("City    ", city);
    return 0;
}
```

Listing 5: Reusable String Capitalization Function

6.3 Output

```
Product : Iphone 15 Pro
Username: Rahul Sharma
```

```
City    : Mumbai
```

Why this refactor is better:

- One function serves **any string input** (constraint satisfied).
- Modifies the string in place (demonstrates pass-by-reference again).
- No duplicated logic—previously similar capitalization code in Task 1 can now reuse this idea.

7 Quick Revision Summary

Task	Function	Passing Style	Key Learning
1	`getUserInitials`	By value (string read-only)	String processing + `toupper()`
2	`addToCart`	By reference (array + `int*`)	Array changes persist
3	`increaseFollowers`	Both styles compared	Copy vs address
4	`formatPrice`	By value in, buffer out	Indian comma formatting
5	`capitalizeFirst`	By reference (in-place)	Refactoring for reusability

Table 1: Summary of Functions, Passing Styles, and Concepts

Golden rule: In C, everything is pass-by-value by default—but passing an **address (pointer)** gives you pass-by-reference behavior, which is how functions like `addToCart` and `capitalizeFirst` modify the caller's data.